Task 1: Deriving the Private Key

```c
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
   char *number_str = BN_bn2hex(a);
   printf("%s 0x%s\n", msg, number_str);
   OPENSSL_free(number_str);
}

int main() {
   BN_CTX *ctx = BN_CTX_new();

   BIGNUM *p = BN_new();
   BIGNUM *q = BN_new();
   BIGNUM *e = BN_new();
   BIGNUM *n = BN_new();
   BIGNUM *phi = BN_new();
   BIGNUM *d = BN_new();
   BIGNUM *p_minus_1 = BN_new();
   BIGNUM *q_minus_1 = BN_new();
   BIGNUM *one = BN_new();

   BN_hex2bn(&p, "5adb09bdbee704d048da9");
   BN_hex2bn(&q, "14738741a1260adf78815");
   BN_hex2bn(&e, "10001");
   BN_dec2bn(&one, "1");

   // Calculate n = p * q
   BN_mul(n, p, q, ctx);

   // Calculate phi(n) = (p-1) * (q-1)
   BN_sub(p_minus_1, p, one);
   BN_sub(q_minus_1, q, one);
   BN_mul(phi, p_minus_1, q_minus_1, ctx);

   // Calculate d = e^(-1) mod phi(n)
   BN_mod_inverse(d, e, phi, ctx);

   // Print results
   printBN("p =", p);
   printBN("q =", q);
```

```
    printBN("e =", e);
    printBN("n =", n);
    printf("Bit length of n: %d\n", BN_num_bits(n));
    printBN("phi(n) =", phi);
    printBN("d (private key) =", d);

    // Cleanup (Free statements omitted)

    return 0;
}
```

Output:

```
seed@cisc468:~/a2$ ./task1
p = 0x05ADB09BDBEE704D048DA9
q = 0x014738741A1260ADF78815
e = 0x010001
n = 0x07421D290018988060A0F421C4BCF19CA09FB066DD
Bit length of n: 163
phi(n) = 0x07421D290018988060A0ED2CDBACFB9BCFA4B45120
d (private key) = 0x0124898EC0F31E5F67D72E4017BDDE4723A3DCB7C1
```

1. What is the bit length of the modulus n?
163 bits

2. Calculate the private key d.
0x0124898EC0F31E5F67D72E4017BDDE4723A3DCB7C1

Task 2: Encrypting a Message

```
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
    char *number_str = BN_bn2hex(a);
    printf("%s 0x%s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *m = BN_new();
```

```
   BIGNUM *c = BN_new();

   // Public key (e, n) from Task 1
   BN_hex2bn(&e, "10001");
   BN_hex2bn(&n, "7421D290018988060A0F421C4BCF19CA09FB066DD");
   // Message "i<3crypto" converted to hex using python script
   BN_hex2bn(&m, "693c3363727970746f");

   // Encrypt: c = m^e mod n
   BN_mod_exp(c, m, e, n, ctx);

   // Print results
   printBN("Message (m) =", m);
   printBN("Public exponent (e) =", e);
   printBN("Modulus (n) =", n);
   printBN("Ciphertext (c) =", c);

   // Cleanup (Free statements omitted)
     return 0;
}
```

Output:

```
seed@cisc468:~/a2$ ./task2
Message (m) = 0x693C3363727970746F
Public exponent (e) = 0x010001
Modulus (n) = 0x07421D290018988060A0F421C4BCF19CA09FB066DD
Ciphertext (c) = 0x01370B0B64176F4EC06B0D590B369857203050E479
```

The ciphertext is **0x01370B0B64176F4EC06B0D590B369857203050E479**

Task 3: Decrypting a Message

```
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
   char *number_str = BN_bn2hex(a);
   printf("%s 0x%s\n", msg, number_str);
   OPENSSL_free(number_str);
}

int main() {
   BN_CTX *ctx = BN_CTX_new();

   BIGNUM *d = BN_new();
```

```
    BIGNUM *n = BN_new();
    BIGNUM *c = BN_new();
    BIGNUM *m = BN_new();

    // Values from Tasks 1 and 2
    BN_hex2bn(&d, "124898EC0F31E5F67D72E4017BDDE4723A3DCB7C1");
    BN_hex2bn(&n, "7421D290018988060A0F421C4BCF19CA09FB066DD");
    BN_hex2bn(&c, "12e6c403952e461afd626e2c81d7b53d0cfe043f3");

    // Decrypt: m = c^d mod n
    BN_mod_exp(m, c, d, n, ctx);

    // Print results
    printBN("Ciphertext (c) =", c);
    printBN("Private key (d) =", d);
    printBN("Modulus (n) =", n);
    printBN("Decrypted message (m) =", m);

    // Print hex string for conversion to ASCII
    char *hex_str = BN_bn2hex(m);
    printf("Hex string for ASCII conversion: %s\n", hex_str);
    OPENSSL_free(hex_str);

    // Cleanup (Free statements omitted)
    return 0;
}
```

Then run python:

```
>> import binascii;
print(binascii.unhexlify(b'73776F726466697368').decode('ascii'))
swordfish
```

The output is **swordfish**.

Task 4: Signing a Message

```c
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
   char *number_str = BN_bn2hex(a);
   printf("%s 0x%s\n", msg, number_str);
   OPENSSL_free(number_str);
}

int main() {
   BN_CTX *ctx = BN_CTX_new();

   BIGNUM *d = BN_new();
   BIGNUM *n = BN_new();
   BIGNUM *m1 = BN_new();
   BIGNUM *m2 = BN_new();
   BIGNUM *s1 = BN_new();
   BIGNUM *s2 = BN_new();

   // Private key d and modulus n from Task 1
   BN_hex2bn(&d, "124898EC0F31E5F67D72E4017BDDE4723A3DCB7C1");
   BN_hex2bn(&n, "7421D290018988060A0F421C4BCF19CA09FB066DD");

   // Message 1: "I owe you $100" in hex
   BN_hex2bn(&m1, "49206f776520796f752024313030");

   // Message 2: "I owe you $500" in hex (changed '1' to '5')
   BN_hex2bn(&m2, "49206f776520796f752024353030");

   // Sign: signature = message^d mod n
   // Note: Signing uses the PRIVATE key exponent d
   BN_mod_exp(s1, m1, d, n, ctx);
   BN_mod_exp(s2, m2, d, n, ctx);

   // Print results
   printf("=== Original Message ===\n");
   printf("Message: \"I owe you $100\"\n");
   printBN("Message (hex) =", m1);
   printBN("Signature =", s1);

   printf("\n=== Modified Message ===\n");
```

```
    printf("Message: \"I owe you $500\"\n");
    printBN("Message (hex) =", m2);
    printBN("Signature =", s2);

    return 0;
}
```

```
seed@cisc468:~/a2$ ./task4
=== Original Message ===
Message: "I owe you $100"
Message (hex) = 0x49206F776520796F752024313030
Signature = 0x032A9C44DE33D475AF40C0F56B8AA024448F25E69F

=== Modified Message ===
Message: "I owe you $500"
Message (hex) = 0x49206F776520796F752024353030
Signature = 0x06614F9C1B3BA583885B1CB089094442EC43B86488
```

1. Generate a signature for the following message (sign this message directly, without hashing it first):

Message: "I owe you $100"
Message (hex) = 0x49206F776520796F752024313030
Signature = 0x032A9C44DE33D475AF40C0F56B8AA024448F25E69F

2. Which exponent did you need to use for signing? (Refer to the course slides.)
The private key d is used for signing.

3. Change one character in the message and sign the modified message. Compare both signatures and describe what you observe.

Changing $100 to $500 actually changed up the message hex only slightly, but the final signature is completely different! This shows that:

- RSA signatures depend on the entire message value
- digital signatures protect against tampering and provides integrity protection

Task 5: Verifying a Signature

```
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
   char *number_str = BN_bn2hex(a);
   printf("%s 0x%s\n", msg, number_str);
   OPENSSL_free(number_str);
}

int main() {
   BN_CTX *ctx = BN_CTX_new();
   BIGNUM *e = BN_new();
   BIGNUM *n = BN_new();
   BIGNUM *m = BN_new();
   BIGNUM *s_valid = BN_new();
   BIGNUM *s_corrupted = BN_new();
   BIGNUM *verified1 = BN_new();
   BIGNUM *verified2 = BN_new();

   // Alice's public key (e, n)
   BN_hex2bn(&e, "010001");
   BN_hex2bn(&n, "ae1cd4dc432798d933779fbd46c6e1247f0cf1233595113aa51b450f18116115");

   // Message M = "Launch a missile." in hex
   BN_hex2bn(&m, "4c61756e63682061206d697373696c652e");
   // Valid signature S
   BN_hex2bn(&s_valid,
"643d6f34902d9c7ec90cb0b2bca36c47fa37165c0005cab026c0542cbdb6802f");
   // Corrupted signature
   BN_hex2bn(&s_corrupted,
"643d6f34902d9c7ec90cb0b2bca36c47fa37165c0005cab026c0542cbdb6803f");

   // Verify: compute S^e mod n and compare to M
   BN_mod_exp(verified1, s_valid, e, n, ctx);
   BN_mod_exp(verified2, s_corrupted, e, n, ctx);

   // Print results for valid signature
   printf("=== Verification with Valid Signature ===\n");
   printf("Message: \"Launch a missile.\"\n");
   printBN("Message (M) =", m);
   printBN("Signature (S) =", s_valid);
```

```
    printBN("S^e mod n =", verified1);

    if (BN_cmp(verified1, m) == 0) {
        printf("Result: SIGNATURE VALID - The signature is indeed from Alice!\n");
    } else {
        printf("Result: SIGNATURE INVALID - The signature is NOT from Alice!\n");
    }

    // Print results for corrupted signature
    printf("\n=== Verification with Corrupted Signature (2F -> 3F) ===\n");
    printBN("Corrupted Signature (S') =", s_corrupted);
    printBN("S'^e mod n =", verified2);

    if (BN_cmp(verified2, m) == 0) {
        printf("Result: SIGNATURE VALID - The signature is from Alice!\n");
    } else {
        printf("Result: SIGNATURE INVALID - The signature is NOT from Alice!\n");
    }

    return 0;
}
```

```
seed@cisc468:~/a2$ ./task5
=== Verification with Valid Signature ===
Message: "Launch a missile."
Message (M) = 0x4C61756E63682061206D697373696C652E
Signature (S) = 0x643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
S^e mod n = 0x4C61756E63682061206D697373696C652E
Result: SIGNATURE VALID - The signature is indeed from Alice!

=== Verification with Corrupted Signature (2F -> 3F) ===
Corrupted Signature (S') = 0x643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6
803F
S'^e mod n = 0x91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
Result: SIGNATURE INVALID - The signature is NOT from Alice!
```

A message is verified if:

Alice signs using her private key $S = M^d \bmod n$, and Bob verifies using Alice's public key $M' = S^e \bmod n$, and M' = M.

We can see that the original message can be indeed verified. This is because:

$ed \equiv 1 \bmod(\phi(n))$, $M' = S^e \bmod n \rightarrow M' = (M^d)^e \bmod n \rightarrow M' = M^{1+k\phi(n)} = M$

The corrupt message would not be verified because even one corrupted bit would cause the signature to be different from our original $M^d \bmod n$. The mod n functions as a scrambling function, so even a tiny change in S produces a completely different result.

Task 6: Cracking an RSA Private Key

To crack an RSA private key, the hardest part of the task is to obtain the set of unique, relatively prime factors p, q based on the modulo n = p * q.

After obtaining p, q, we compute the quotient $\phi(n) = (p - 1)(q - 1)$
Then find the private exponent d as the modular inverse of e modulo ϕ(n), which means we solve $e \times d \equiv 1 \, mod \, \phi(n)$ using the extended Euclidean algorithm.

There exists a couple libraries that does this efficiently. One library that I found was called yafu (GitHub: https://github.com/bbuhrow/yafu)

Yafu is capable of finding the prime factors effectively because it has strategies like Trial division, Rho (Pollard rho), ECM (Elliptic Curve Method), and SIQS (Self-Initializing Quadratic Sieve) coded in its library.

1.

```
python3 -c "print(0x7c5cfe617c286a27ffc10ecf88a8d35ebbf1e30320af"
```

Outputs us the n value in decimal format:
465298183837103746727116591565054904463785634868184 79

Then we pass it into YAFU to find the prime factors

```
echo "factor(465298183837103746727116591565054904463785634868184 79)" |
./yafu
```

This outputs a long log that utilizes the different methods mentioned above.

Then I have made a python script that is capable of decode the private key d using e, n, p, q.

```
def extended_gcd(a, b):
    """Extended Euclidean Algorithm: returns (gcd, x, y) where a*x + b*y = gcd."""
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    return gcd, y1 - (b // a) * x1, x1
```

```python
def mod_inverse(e, phi):
    """Compute d = e^(-1) mod phi using Extended Euclidean Algorithm."""
    gcd, x, _ = extended_gcd(e % phi, phi)
    if gcd != 1:
        raise ValueError("Modular inverse does not exist")
    return x % phi


def crack_rsa_key(name, e, n, p, q):
    """
    Given public key (e, n) and its prime factors (p, q) from YAFU,
    compute the private exponent d.
    """
    assert p * q == n, "ERROR: p * q != n"
    phi_n = (p - 1) * (q - 1)
    d = mod_inverse(e, phi_n)

    # Verify: encrypt then decrypt a test message
    m = 42
    assert pow(pow(m, e, n), d, n) == m, "ERROR: verification failed"

    print(f"--- {name} ---")
    print(f"  n = 0x{n:x}  ({n.bit_length()} bits)")
    print(f"  p = 0x{p:x}")
    print(f"  q = 0x{q:x}")
    print(f"  d = 0x{d:x}")
    print()

    return d
```

Found key: d = 0x39c1e2ae99c017554e557582b85dca21843c2bfcddd1

2. Similarly

```
python3 -c
"print(0x1e6f1558fe63761406be065a9e07b060f8360b2725b09f4071"
```

Outputs us the n value in decimal format:
191036808839073752998059970737347860342917676432172452364401

Then we pass it into YAFU to find the prime factors

```
echo
"factor(191036808839073752998059970737347860342917676432172452364401)"
| ./yafu
```

Outputs us prime:
335403766841212825761913864577
569572639682113597061986473713

Found key: d = 0x6c5dff15b43d54e65a5ba9243985ca4e0f3c03ffcca3e2801

3. Similarly

```
python3 -c "print(a962e5e6455feec804c6e6faa62b66a50675a8
24e5e73ac03b009a7d36abb557c14418444d45"
```

Outputs us the n value in decimal format:
215653767958005306447782312224930614867177974189983834711311896825161936664158380290141258 93

Then we pass it into YAFU to find the prime factors

```
echo
"factor(21565376795800530644778231222493061486717797418998383471131189
6825161936664158380290141258 93)" | ./yafu
```

Outputs us prime:
434337469789106547420003184262530341009384571 3
496512004968662817470111771573609666075435986 1

Found key: d =
0xa55cb36c36560d5a9faf3e577c59ea7f38eb2a803be7f5f6da9c40a8d87298e726f02815f301

Runtime script:

```
if __name__ == "__main__":
    print()
    print("Task 6: RSA Private Key Cracking (factors found via YAFU)")
```

```
    print("Full YAFU logs: task6_yafu_log.txt")

    d1 = crack_rsa_key(
        name="Key 1 (175 bits, 53 digits — YAFU: ~1.5s via SIQS)",
        e=0x10001,
        n=0x7c5cfe617c286a27ffc10ecf88a8d35ebbf1e30320af,
        p=15831485998638128003212378 3,
        q=29390682837803734573645711 3,
    )

    d2 = crack_rsa_key(
        name="Key 2 (197 bits, 60 digits — YAFU: ~3.2s via SIQS)",
        e=0x10001,
        n=0x1e6f1558fe63761406be065a9e07b060f8360b2725b09f4071,
        p=335403766841212825761913864577,
        q=569572639682113597061986473713,
    )

    d3 = crack_rsa_key(
        name="Key 3 (304 bits, 92 digits — YAFU: ~12s via SIQS)",
        e=0x10001,
n=0xa962e5e6455feec804c6e6faa62b66a50675a824e5e73ac03b009a7d36abb557c14418444d45,
        p=434337469789106547420003184262530341009384571 3,
        q=496512004968662817470111771573609666075435986 1,
    )
```

4.
I first tried cracking the keys using the SymPy library's factorint in Python on my Mac (MacBook M1 Pro, 32GB RAM), but it was slow for the larger ones, so I switched to running Yafu on my PC (AMD Ryzen 9 9950X3D, RTX5070Ti, 96GB RAM).

The first key (175 bits) factored in ~1.5s via SIQS, the second (197 bits) in ~3.2s, and the third (304 bits) in ~12s. The logs show that solving the private key mostly requires SIQS after initial trial/ECM.

Cracking time grew super fast (even though my PC stack was qutie good). The time it took roughly increased 5-6 times as bits roughly doubled (from 175 to 304 bits). This is because RSA security relies on factoring huge n = p * q being exponentially hard; bigger

keys mean larger primes, so sieving methods like SIQS need way more trial relations and matrix solves, turning seconds into hours or days beyond 512 bits.

Seed lab terminal output screenshot:

```
seed@cisc468:~/a2$ python3 task6.py

  Task 6: RSA Private Key Cracking (factors found via YAFU)
  Full YAFU logs: task6_yafu_log.txt

  --- Key 1 (175 bits, 53 digits — YAFU: ~1.5s via SIQS) ---
    n = 0x7c5cfe617c286a27ffc10ecf88a8d35ebbf1e30320af  (175 bits)
    p = 0x82f479c1b700bb6c34eb87
    q = 0xf31d31177f86471723fb99
    d = 0x39c1e2ae99c017554e557582b85dca21843c2bfcddd1

  --- Key 2 (197 bits, 60 digits — YAFU: ~3.2s via SIQS) ---
    n = 0x1e6f1558fe63761406be065a9e07b060f8360b2725b09f4071  (197 bits)
    p = 0x43bbf7e497406aff037318181
    q = 0x7306371ea9521f9c3150356f1
    d = 0x6c5dff15b43d54e65a5ba9243985ca4e0f3c03ffcca3e2801

  --- Key 3 (304 bits, 92 digits — YAFU: ~12s via SIQS) ---
    n = 0xa962e5e6455feec804c6e6faa62b66a50675a824e5e73ac03b009a7d36abb557c14418444d45  (304 bits)
    p = 0xc2c37f0d5c3a095bf6db0c7be3b8418fc3ecd1
    q = 0xdea4c86c84e449d53d5298edd6b0cf21096635
    d = 0xa55cb36c36560d5a9faf3e577c59ea7f38eb2a803be7f5f6da9c40a8d87298e726f02815f301
```